

Biped Locomotion Challenge — Simulation-Only Walking Control

Report for ECE_DK801 Robotics Systems I, 2025–2026

Team 5 — Mpantekas Nikolaos (up1092562), Stathopoulos Stavros (up1101069)

Abstract

We develop a torque-only walking controller for the 29-DoF Unitree G1 humanoid in MuJoCo. Locomotion is generated by a Divergent-Component-of-Motion (DCM / capture-point) pattern generator built on the Linear Inverted Pendulum Model (LIPM), and realised by an online whole-body inverse-dynamics Quadratic Program (WBQP) that resolves the CoM, torso, swing-foot and posture tasks into the 29 joint torques subject to the full floating-base dynamics, friction cones and actuator limits. The robot is never pinned, welded or teleported; the only runtime output is the 29-dimensional `data.ctrl` torque vector. Measured on headless 30 s runs, the controller walks 66 steps (≈ 2.0 m) without falling on flat ground, a $\pm 3^\circ$ ramp pair and a low staircase, with zero QP failures, tracks the reference DCM laterally to 15 mm RMS, and survives sagittal base-velocity disturbances of at least 0.70 m/s.

1 System Overview and Methodology

The plant is the Unitree G1 (`g1.29dof.xml`): a floating base (6 unactuated DoF) plus 29 hinge joints, each driven by a pure-torque `<motor>` actuator. Verified from the loaded model: $n_q = 36$, $n_v = 35$, $n_u = 29$, total mass 35.11 kg. The control loop runs at 500 Hz ($dt = 2$ ms). Foot contact is modelled by four corner spheres per ankle-roll link (17 cm $\times$ 6 cm: heel -5 cm, toe $+12$ cm, half-width 3 cm); the floor has friction coefficient $\mu_{\text{floor}} = 1.0$. Actuator torque limits read from the MJCF are summarised in Table 1.

Table 1: Robot actuation (per-joint torque limits, from `jnt_actfrcrange`).

Joint group	count	τ_{max} [Nm]
Hip pitch / roll / yaw	6	88
Knee	2	139
Ankle pitch / roll	4	50
Waist yaw / roll / pitch	3	88 / 50 / 50
Shoulder / elbow	12	25
Wrist roll / pitch / yaw	6	25 / 25 / 5

The per-tick pipeline (`scripts/run.dcm.walk.py`) is a strict feed-forward cascade of three layers; no quantity in `qpos/qvel` is overwritten during the run (the home crouch is written once, as an initial condition):

```
(L1)   $s \leftarrow \text{StateEstimator.update}(d)$ 
(L2)  (refs, info)  $\leftarrow \text{DCMWalkingGait.update}(s, dt)$ 
(L3)   $\tau \leftarrow \text{WholeBodyQP.compute}(d, \text{refs})$ 
       $d.\text{ctrl} \leftarrow \tau$ ;  mj_step(m, d)
```

Design rationale. The G1’s feet are small and its ankle-roll torque is weak (± 50 Nm), so lateral balance *cannot* be held by ankle CoP modulation alone—it must come from *where the swing foot is placed*. This motivates the DCM/capture-point approach: the capture point dictates where to step to arrest the falling CoM, while the DCM feedback law dictates the ground reaction (ZMP) to command between steps. The two together produce a stable walking limit cycle.

1.1 Layer 1: State estimation

`StateEstimator` (`robot_model.py`) reads MuJoCo state each tick and produces: CoM position c from `subtree_com[0]`; CoM velocity $\dot{c} = J_{\text{com}}\dot{q}$ via `mj_jacSubtreeCom`; base roll/pitch/yaw from the base quaternion; foot poses from the ankle-roll body frames; and per-foot normal force, obtained by summing `mj_contactForce` over active contacts whose geoms belong to that foot and thresholding at 15 N to give a boolean contact and a support phase (DOUBLE/LEFT/RIGHT/FLIGHT).

1.2 Layer 2: DCM / capture-point gait generator

DCMWalkingGait (`dcm_gait.py`) builds on the LIPM with constant CoM height z_c (measured $z_c = 0.679\text{ m}$, giving $\omega = \sqrt{g/z_c} = 3.80\text{ rad/s}$):

$$\xi = c + \dot{c}/\omega, \quad \dot{\xi} = \omega(\xi - p), \quad \dot{c} = \omega(\xi - c), \quad (1)$$

where ξ is the DCM (capture point) and p the ZMP/CoP. A four-state FSM (Table 2) sequences double- and single-support phases.

Table 2: Gait FSM (timing as configured in `run_dcm_walk.py`).

State	dur. [s]	contacts	role
STAND	1.2	both	settle, CoM at foot midpoint
START	1.0	both	one-off weight shift onto 1st stance
DS	0.12	both	transfer weight to next stance
SS	0.30	stance	swing foot to capture target

Continuous DCM reference. The end-of-step DCM is made continuous across steps by a backward recursion over an $N=6$ -step preview of *absolute* footsteps ($x_k = x_0 + k\ell$, $y_k = \pm w/2$ by parity):

$$\xi_{\text{eos},N} = p_N, \quad \xi_{\text{eos},i-1} = p_i + (\xi_{\text{eos},i} - p_i)e^{-\omega T_{ss}}, \quad (2)$$

which for a periodic gait converges to the classic $\pm d \tanh(\omega T/2)$ sway. Within a step the reference DCM and reference CoM follow

$$\xi_{\text{ref}}(\tau) = p_{st} + (\xi_{\text{eos}} - p_{st})e^{\omega(\tau - T_{ss})}, \quad \dot{c}_{\text{ref}} = \omega(\xi_{\text{ref}} - c_{\text{ref}}), \quad (3)$$

and the CoM acceleration feed-forward handed to the QP is $\ddot{c}_{\text{ref}} = \omega^2(c_{\text{ref}} - p_{\text{ref}})$. The stance ZMP is laterally anchored to the ideal centreline foothold ($y = \pm d$) so the reference trajectory does not veer; the forward coordinate tracks the actual foot so progress accumulates.

ZMP feedback (Englsberger). The measured DCM error is mapped to a commanded ZMP (diagnostic / CoP clamp), $p_{\text{cmd}} = p_{\text{ref}} + (1 + k_{\text{dcm}}/\omega)(\xi - \xi_{\text{ref}})$, $k_{\text{dcm}} = 2.5$, clamped to the support polygon shrunk by 2 cm.

Capture-point foot placement. During single support the predicted end-of-step DCM steers the landing site, bounded about the absolute nominal foothold:

$$\xi_{\text{pred}} = p_{st} + (\xi - p_{st})e^{\omega(T_{ss} - \tau)}, \quad (4)$$

$$\Delta y = \text{clip}(k_{\text{cap}}(\xi_{\text{pred},y} - \xi_{\text{eos},y}), \pm 6\text{ cm}), \quad (5)$$

with $k_{\text{cap}} = 0.8$ (and an analogous, clamped sagittal correction $\pm 4\text{ cm}$). Clamping to the absolute foothold is essential: an unbounded correction chases the absolute DCM, creating positive feedback that walks the feet to the centreline until they cross. The swing foot follows a minimum-jerk (quintic) xy trajectory with a sinusoidal z arch (step height 15 cm).

1.3 Layer 3: Whole-body inverse-dynamics QP

WholeBodyQP (`wbqp.py`) solves, every 2 ms, a QP in $x = [\ddot{q} (n_v), \lambda \text{ (3 per contact point)}]$:

$$\min_x \sum_t w_t \|J_t \ddot{q} - a_t^*\|^2 + r_{\ddot{q}} \|\ddot{q}\|^2 + r_f \|\lambda\|^2 \quad (6)$$

subject to floating-base dynamics $M_{[6]} \ddot{q} + h_{[6]} = J_{c,[6]}^\top \lambda$; stance no-acceleration (Baumgarte-damped) $J_c \ddot{q} = -k_c J_c \dot{q}$; per-point friction pyramids $|f_{x,y}| \leq \mu f_z$, $f_z \geq f_{z,\min}$ ($\mu = 0.7$, $f_{z,\min} = 2\text{ N}$); and torque limits $-\tau_{\max} \leq M_{[6]} \ddot{q} + h_{[6]} - J_{c,[6]}^\top \lambda \leq \tau_{\max}$. Tasks and weights are listed in Table 3; each uses an acceleration-level PD law $a^* = k_p e + k_d \dot{e}$ (plus $\ddot{c}_{\text{ref}}$ for the CoM). Because `quadprog` has no native equalities, the two equality blocks are folded in as a stiff least-squares penalty ($w_{eq} = 10^4$), enforcing them to $\sim 10^{-4}$. The actuated torques are recovered from the lower dynamics rows, $\tau = (M \ddot{q} + h - J_c^\top \lambda)_{[6:]}$, and clipped; if the solve fails the controller falls back to gravity compensation $\tau = h_{[\text{act}]}$. There is deliberately *no* downstream joint-PD: the QP already incorporates M and h , so an extra PD would double-count them.

2 Experimental Procedure

Environment. MuJoCo 3.8.1; NumPy; `qp-solvers+quadprog` as the dense QP backend. The model is loaded from `assets/unitree_g1/`; the integrator timestep is forced to $1/\text{rate} = 2\text{ ms}$. Runs are headless (`MUJOCO_GL=osmesa`), so results are renderer-independent.

Table 3: WBQP task set and weights (as configured for walking).

Task	gains (k_p, k_d)	weight
CoM (+accel. FF)	[90, 80, 90], [28, 20, 19]	30
Torso/pelvis orientation	250, 30	12
Swing-foot position	400, 40	50
Swing-foot orientation	80, 18	3
Posture (home)	40, 9	0.1 (hip-yaw $\times 90$)

Initial condition. `RobotModel.set_home` writes a bent-knee crouch (hip pitch -0.40 , knee $+0.80$, ankle pitch -0.40 rad; flat feet because the three sum to zero) at the analytically computed resting pelvis height (≈ 0.744 m, 2 mm clearance) and zeroes all velocities. The gait is then `reset` from the first state estimate.

Scenes. Three MJCF worlds share `g1_29dof.xml`: `scene.xml` (flat plane); `scene_tilted.xml` (a $+3^\circ$ uphill box ramp at $x=0.40$ followed by a -3° downhill ramp at $x=1.48$, each 1.1 m long); and `scene_stairs.xml` (nine 14 cm treads rising then falling in 5 mm increments). Terrain geoms are tagged render group 2, which the ray-cast terrain sensor (`terrain.py`) keys on.

Protocol. For each scene we ran

```
python scripts/run_dcm_walk.py --headless --duration 30 --scene <scene>
```

with the stable default gait ($\ell=0.03$ m, $w=0.22$ m, $T_{ss}=0.30$ s, $T_{ds}=0.12$ s, $k_{dcm}=2.5$, $k_{cap}=0.8$). Each 0.25 s the script logs FSM state, step count, stance, CoM, DCM, commanded ZMP, torso roll/pitch, per-foot force, peak torque and torque-saturation ratio; we parsed these 120-sample logs for the aggregate metrics in Section 4. A *disturbance* sweep applied an instantaneous horizontal base velocity (`--kick V --seed 1`) at $t=0$ and recorded survival over 15 s for $V \in \{0.10, \dots, 0.70\}$ m/s. A separate instrumented driver logged per-tick trajectories (12 s, flat) to produce Figs. 1–2. Termination criteria: “fell” if pelvis height < 0.45 m; the gait self-aborts to a stand if torso tilt exceeds 0.5 rad.

3 Analysis

Why the limit cycle is stable. The DCM is the unstable eigenmode of the LIPM ($\dot{\xi} = \omega(\xi - p)$); the remaining CoM mode is stable and simply follows ξ . Control therefore reduces to regulating one unstable first-order state per axis. The reference ξ_{ref} is built so that each step’s initial DCM equals the previous step’s end-of-step DCM (the backward preview recursion), making the planned trajectory a fixed point of the step map; for the periodic footstep pattern this is exactly the $\pm d \tanh(\omega T/2)$ lateral sway. Disturbances are rejected on two channels: (i) the WBQP CoM task servos the *measured* CoM onto c_{ref} , which removes slow drift that pure acceleration feed-forward cannot hold; and (ii) capture-point foot placement moves the next foothold under the predicted end-of-step DCM, which is the only authority strong enough for the lateral plane given the weak ankles.

Why the QP suffices as the tracker. Solving at the acceleration/force level lets one convex program reconcile coupled objectives (CoM, upright torso, swing-foot placement) against the hard physics (contact non-acceleration, friction, τ_{max}). The CoM weight (30) exceeds the posture weight (0.1) by >2 orders of magnitude, so balance tracks tightly while posture only regularises the redundant DoF; the hip-yaw posture multiplier ($\times 90$) keeps the legs from toeing in and crossing. Coriolis task-bias terms ($\dot{J}\dot{q}$) are dropped (the gait is near-quasi-static), validated empirically by **zero** QP failures.

Observed behaviour. Fig. 1 shows the lateral plane settling within the first step into a clean periodic sway: ξ_y tracks ξ_y^{ref} , the CoM c_y lags inside it as the LIPM predicts, and the per-foot normal force alternates between ~ 0 N and 300–460 N (left/right single support)—the robot genuinely unloads each swing foot. Fig. 2 shows the forward CoM advancing monotonically while locked to its reference (6 mm RMS), torso roll/pitch bounded inside ± 0.10 rad, and torque saturation mostly < 0.5 , spiking to 1.0 only at isolated knee-loading instants. The DCM-driven CoP commands stay inside the shrunk support polygon.

Terrain. On the ramps and stairs the sagittal capture correction (± 4 cm) and a CoM-height target that tracks mean foot height let the gait adapt without re-tuning. The DCM gait deliberately does *not* consume the `terrain.py` ray sensor for landing height: feeding measured foot heights back coupled the sagittal and lateral planes in a way the point-mass LIPM does not model and destabilised the lateral catch, so terrain is handled implicitly.

4 Results

Table 4 reports the measured outcome of the 30 s headless runs. All three scenes complete the full duration with no fall, 66 steps and ≈ 2.0 m of forward travel (step length $\ell = 0.03$ m $\times 66 \approx 2.0$ m, consistent with the absolute footstep plan). Cadence is $1/(T_{ss}+T_{ds}) = 2.38$ steps/s (≈ 0.067 m/s). The QP solved on every one of the ~ 15 000 ticks per run without a single failure, so the gravity-compensation fallback was never exercised. Lateral excursion stays within 4 cm and CoM height within a 4 cm band, confirming the constant-height LIPM assumption holds in closed loop.

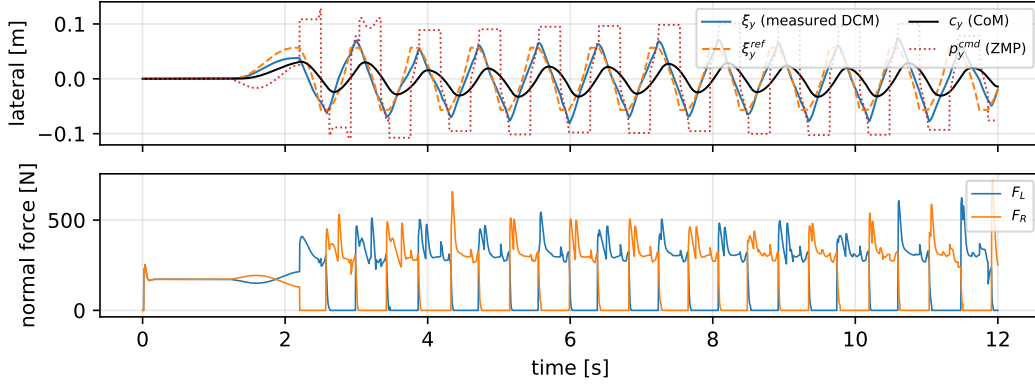


Figure 1: Lateral plane (flat ground). *Top*: measured DCM ξ_y tracks the reference, the CoM c_y swings inside it, and the commanded ZMP p_y^{cmd} stays bounded—the $\pm d \tanh(\omega T/2)$ limit cycle. *Bottom*: per-foot normal force confirms clean single-support alternation.

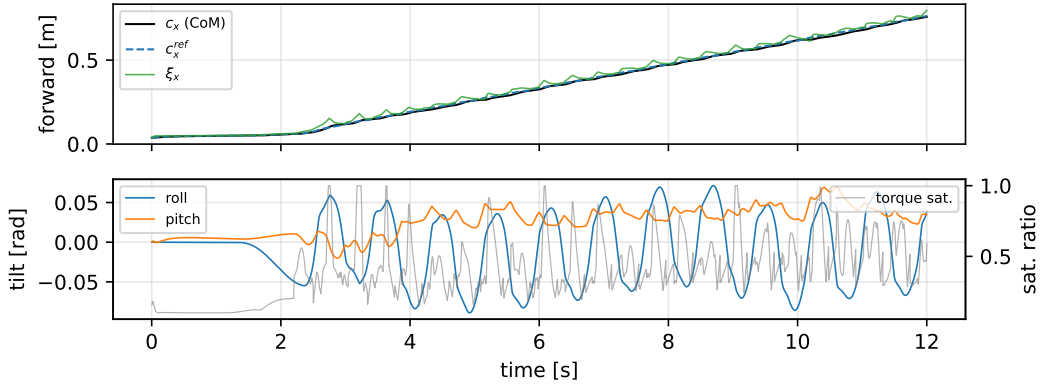


Figure 2: Sagittal/attitude (flat ground). *Top*: forward CoM c_x locked to its DCM-consistent reference (ξ_x leads). *Bottom*: torso roll/pitch stay within ± 0.10 rad while the worst-case torque-saturation ratio is mostly below 0.5.

Tracking accuracy. From the per-tick instrumented run (flat, 12s): the measured DCM tracks its reference to **15.0 mm RMS** laterally, and the forward CoM tracks its DCM-consistent reference to **6.2 mm RMS**. These small residuals—against a 22 cm stance width and a 3 cm step—explain the clean, repeatable limit cycle in Fig. 1.

Disturbance rejection. In a base-velocity kick sweep (instantaneous horizontal impulse at $t=0$, 15 s flat run, seed 1) the robot survived *every* tested magnitude $V \in \{0.10, 0.15, 0.25, 0.45, 0.55, 0.70\}$ m/s with no fall and essentially unchanged travel, the capture-point step adjustment absorbing the perturbation within a few steps; 0.70 m/s is thus a conservative lower bound on the recoverable push.

Coverage vs. assignment methods. Implemented: LIPM, DCM/capture-point planning and stepping, CoM trajectory planning, weight shifting, FSM phase management, ZMP feedback, and whole-body (inverse-dynamics) control with friction and torque constraints. The 6-step DCM preview plays the planning-ahead role of MPC but is an analytical closed-form recursion, not a receding-horizon optimisation; a whole-body *IK* module exists (`wbik.py`) but the walking path uses inverse dynamics directly. Push recovery is limited to capture-point step correction plus a tilt-triggered abort to standing; explicit terrain-height sensing exists but is intentionally not coupled into the DCM gait.

Limitations. The CoM-height target only rises (never lowers), so long steep descents would eventually over-extend the legs; momentary torque saturation on knee loading indicates little headroom for larger/faster steps; and lateral stability rests on the LIPM point-mass abstraction, whose neglected rotational coupling the torso task must actively suppress. All numbers above were produced by the Section 3 commands against the committed code; figures are regenerated by `report/make_figures.py`.

Table 4: Measured results, 30 s headless runs (parsed from the diagnostic logs).

Metric	Flat	Ramps $\pm 3^\circ$	Stairs
Survived [s]	30.0	30.0	30.0
Fell	no	no	no
Steps	66	66	66
Forward travel [m]	2.00	2.01	2.00
Lateral drift $ y $ [m]	0.040	0.032	0.036
Final pelvis z [m]	0.761	0.770	0.759
Max $ \text{roll} $ [rad]	0.10	0.07	0.09
Max $ \text{pitch} $ [rad]	0.07	0.06	0.05
Peak torque [Nm]	64	60	54
Mean / max sat. ratio	0.47 / 1.0	0.38 / 1.0	0.40 / 1.0
QP failures	0	0	0